



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

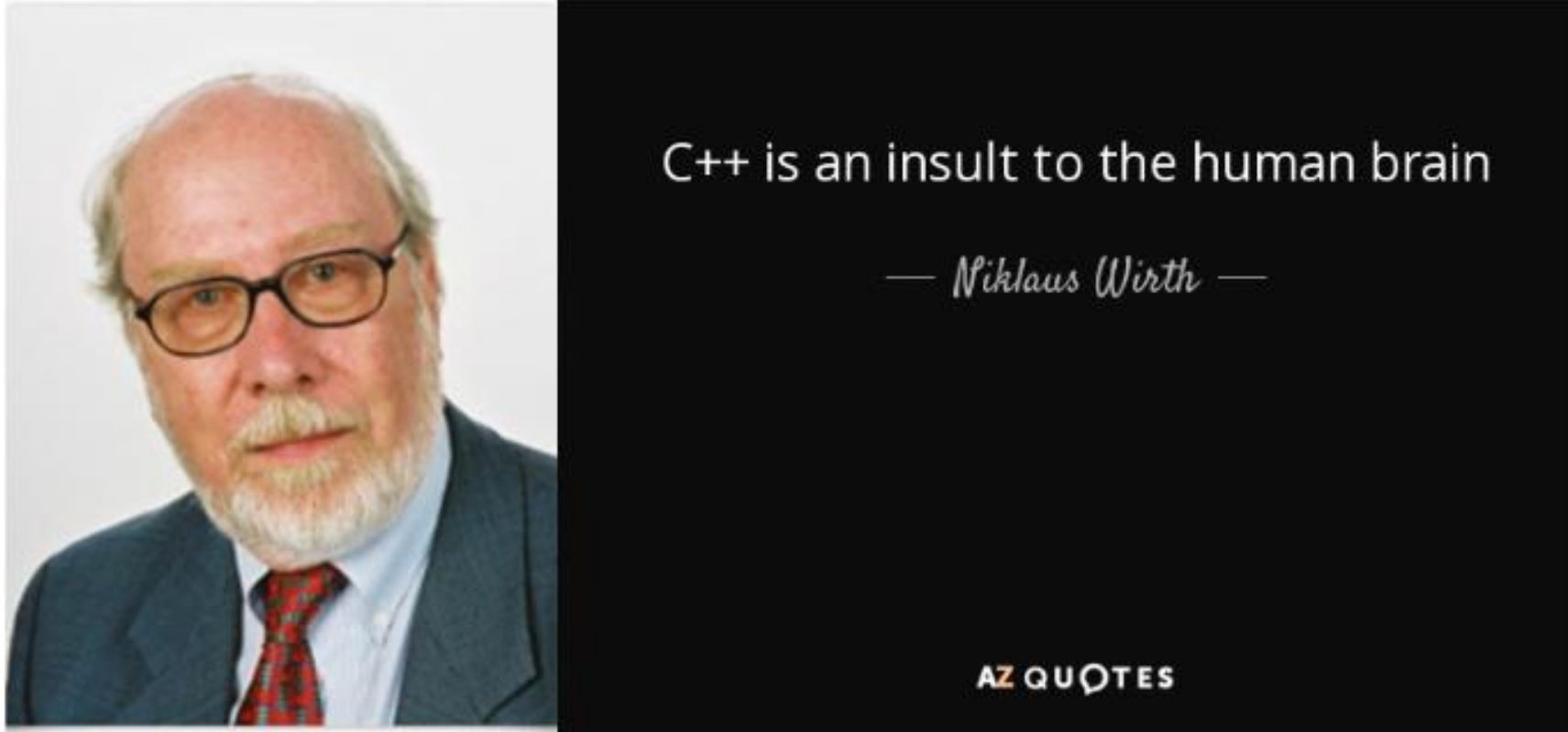
Game Engine Architectures

Fachhochschule Oberösterreich

C++ Background

Basics

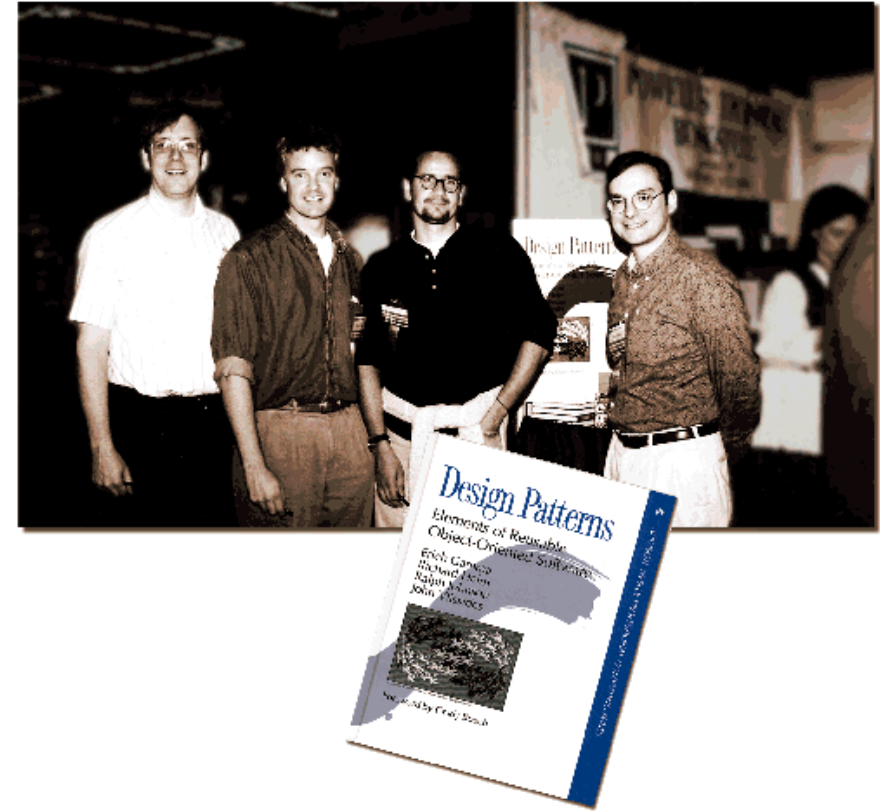
C++ is very complex...



Basics

C++ History

- Bjarne Stroustrup (AT&T Bell Labs, 1982), "C with Classes"
- Implementation as commercial product (1985)
- Borland Turbo C++ (additional libraries, 1990)
- Design Patterns (1994/1995)
- C++ ISO/IEC 14882:1998 Standard (1998)
- C++0x (2005, 2011)
- C++20 (2020)



Basics

C++ Advantages vs. other languages

- Object-oriented
- High speed
- Full compilation to machine code
- Pointer support
- Close-to-hardware

Basics

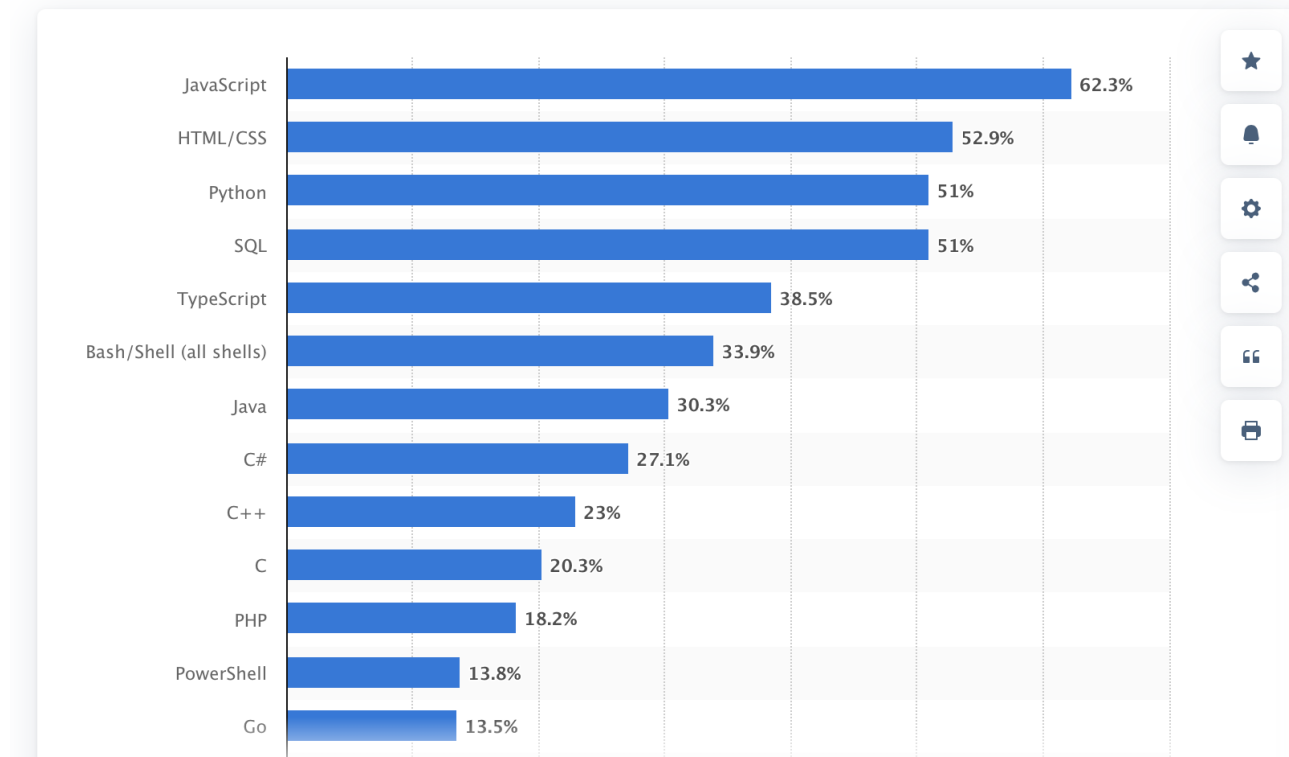
C++ Applications

- Operating Systems (Windows, Mac OS X)
- Large GUI Applications (Photoshop, MS Office, Visual Studio, etc.)
- Web Browsers (Firefox, Internet Explorer, partly Chrome)
- Embedded Systems (smartwatches, medical equipment, etc.)
- Compilers
- Databases
- Network Devices
- Games

Basics

C++ is very complex...

Most used programming languages among developers worldwide as of 2024

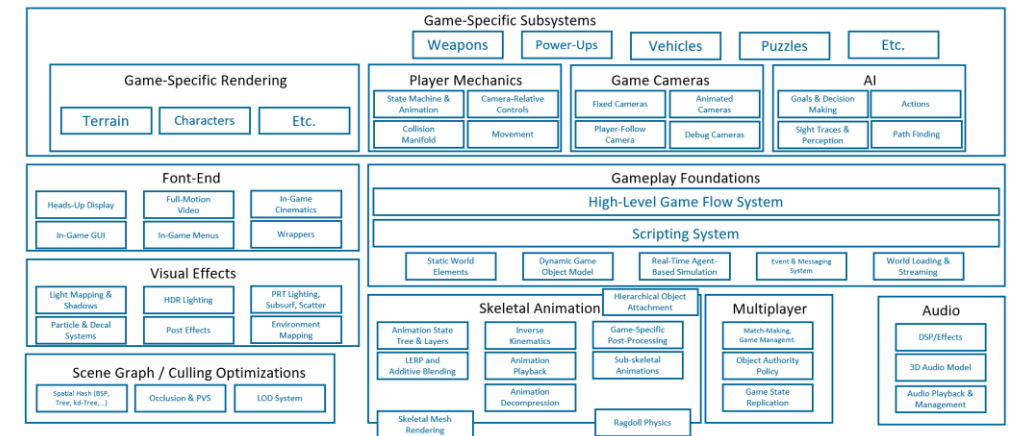


C++ Basics

Overview

The Computer Scientist's Mind

- The programs themselves are translated into machine code
- The machine code is the atomic language of a particular computer
- Computers are both complex and entirely simple
- **Today, we will learn about the basic concepts of C++**



```
// Called to bind functionality to input
void APlayerPawn::SetupPlayerInputComponent(UInputComponent* PlayerInputComponent)
{
    Super::SetupPlayerInputComponent(PlayerInputComponent);
}

00000000: 65 6C 6C 6F 57 6F 72 6C 16 01 00 10 6A 61 76 61 : elloWorld@ java
00000000: 2F 6C 61 6E 62 2F 4F 62 16 65 63 74 01 00 10 6A : /lang/Object@ j
00000000: 61 76 61 2F 6C 61 6E 67 12F 53 79 73 74 65 6D 01 : ava/lang/System@
00000000: 00 03 6F 75 74 01 00 15 14C 6A 61 76 61 2F 69 6F : vout@ java/io
00000000: 2F 50 72 69 6E 74 53 74 12 65 61 6D 3B 01 00 13 : /PrintStream@ !!
00000000: 6A 61 76 61 2F 69 6F 2F 150 72 69 6E 74 53 74 72 : java/io/PrintStr
00000000: 65 61 6D 01 00 07 70 72 16 6E 74 6C 6E 01 00 15 : eam@ println@ $
```

Basics

C++ Hello World

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello Hagenberg!\n";
7  }
```



C:\ Microsoft Visual Studio-Debugging-Konsole

Hello Hagenberg!

Basics

C++ Hello World

Preprocessor directive: `#include` tells preprocessor to add another file (called a “header”) into this file.

The preprocessor copy/pastes this code into the current file.

Here: `iostream`, which allows in- and output of data

```
1  #include <iostream>
2  using namespace std;
4  int main()
5  {
6      cout << "Hello Hagenberg!\n";
7  }
```

Basics

C++ Hello World

Preprocessor directive:

Without the `#include`, the object “cout” is not found and cannot be used

```
1  using namespace std;
2
3  int main()
4  {
5      cout << "Hello Hagenberg!\n";
6  }
```

Der Bezeichner ""cout"" ist nicht definiert.
[Online suchen](#)
[Mögliche Korrekturen anzeigen \(Alt+Eingabe oder Strg+.\)](#)

Basics

C++ Hello World

Namespace:

using namespace tells the compiler to look for a section and directly allow to use the objects (i.e., variables, objects, functions, etc.) in that namespace

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello Hagenberg!\n";
7  }
```

Basics

C++ Hello World

Missing “using namespace”:

Without, the object “cout” is not found and cannot be used

```
1  #include <iostream>
2
3  int main()
4  {
5      cout << "Hello Hagenberg!\n";
6  }
```

Der Bezeichner ""cout"" ist nicht definiert.

[Online suchen](#)

[Mögliche Korrekturen anzeigen](#) (Alt+Eingabe oder Strg+.)

Basics

C++ Hello World

Missing “using namespace”:

We could still use the namespace directly to not run into an error!

```
1  #include <iostream>
2
3  int main()
4  {
5      std::cout << "Hello Hagenberg!\n";
6  }
```


Basics

C++ Hello World

Main Method:

Every program MUST contain EXACTLY ONE main method that tells the compiler “where to start” It should return a number (int)

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello Hagenberg!\n";
7  }
```

Basics

C++ Hello World

Source Code:

The source of this program just tells the compiler to use the “cout” object in the namespace std and “give it” the value “Hello Hagenberg!\n”

```
1  #include <iostream>
2  using namespace std;
3
4  int main()
5  {
6      cout << "Hello Hagenberg!\n";
7  }
```

C++ Data Types

- **int (Integer)**, 4 bytes; -2147483648 to 2147483647
 - **char (Character)**, 1 byte; signed: -127 to 127, i.e., ASCII values
 - **bool (Boolean)**, 1 byte; true | false
 - **float (Floating point number)**, 4 bytes; $1.175494351 \text{ E} - 38$ to $3.402823466 \text{ E} + 38$
 - **double (Double floating point number)**, 8 bytes; $2.250... \text{ E} - 308$ to $1.797... \text{ E} + 308$
 - **void (Valueless)**, zero bytes/bits
 - **wchar_t (Wide Character)**, 2 or 4 bytes
- As C++ is close to hardware, the exact specs can vary depending on the compiler for a particular machine!

Basics

C++ Grammar

- **Identifier:** names for objects, [1-9,a-z,A-Z,_], not starting with a number
- **Keywords:** int, class, if, while, namespace, new, public, template, etc.
- **Literals:** 5, 5.5, 5.5f, 5.5F, 5.5L, '\n', "hello, Hagenberg!", true, false, etc.
- **Operators and Limiters:** +, -, ++, --, *, /, &, +=, etc.
- **Comments:** //, /* */

Op	Name	Example	Op	Name	Example
++	Postincrement	Value++	*	Multiply	Expr*expr
++	Preincrement	++value	/	Divide	Expr/expr
--	Postdecrement	Value--	%	Modulo	Expr%expr
--	Predecrement	--value	+	Binary Add	Expr+expr
+	Unary add	+expr	-	Binary Subtract	Expr-expr
-	Unary subtract	-expr	!	Logical not	!expr
&&	Logical and	Expr && expr	&	Bitwise and	Expr & expr
	Logical or	Expr expr		Bitwise or	Expr expr
~	Bit complement	~expr	^	Bitwise xor	Expr ^ expr
<	Less	Expr < expr	<<	Shift-left	Expr << expr
>	Greater	Expr > expr	>>	Shift-right	Expr >> expr
==	Equals	Expr == expr	<=	Less or equal	Expr <= expr
!=	Not Equals	Expr != expr	>=	Greater or equal	Expr >= expr

Basics

C++ Grammar, Limiters

- **;** **End of Instruction:** in theory, no newline needed...
- **,** **Splitting arguments:** parameters, values in an array, etc.
- **()** **Isolation:** function parameter lists, expressions, priorities
- **[]** **Arrays:** collection of elements of the same type
- **:** **Label:** switch, goto (don't use!)
- **{ }** **Block:** creating subsections in code
- **.** **Member:** Accessing class elements
- **->** **Pointer:** stores the memory address as its value

Void expression
(doing absolutely nothing)

```
int main()  
{  
;  
int b = 1;  
int d = 3.14f;  
char a = 'a';  
d *= 2;  
a = b = 0;  
}
```

Assignment:

An integer variable with the value 1

```
int main()  
{  
    ;  
    int b = 1;  
    int d = 3.14f;  
    char a = 'a';  
    d *= 2;  
    a = b = 0;  
}
```


Basics

C++ Expressions

Assignment:

An integer variable that is assigned a float value?

d=3

```
int main()  
{  
    ;  
    int b = 1;  
    int d = 3.14f;  
    char a = 'a';  
    d *= 2;  
    a = b = 0;  
}
```

Assignment:

A character variable, which value?
a = 97 (ASCII)

```
int main()  
{  
    ;  
    int b = 1;  
    int d = 3.14f;  
    char a = 'a';  
    d *= 2;  
    a = b = 0;  
}
```

What is the result of this expression?
d = 6

```
int main()  
{  
    ;  
    int b = 1;  
    int d = 3.14f;  
    char a = 'a';  
    d *= 2;  
    a = b = 0;  
}
```

What is the result of this expression?

a=0;

b=0;

```
int main()  
{  
    ;  
    int b = 1;  
    int d = 3.14f;  
    char a = 'a';  
    d *= 2;  
    a = b = 0;  
}
```

Blocks are indicated with {}

```
int main()
{
    int i = 0;
    {
        int i = 1;
        cout << i << endl;
    }
    cout << i << endl;
}
```

What is the output of this program?

1
0

```
int main()
{
    int i = 0;
    {
        int i = 1;
        cout << i << endl;
    }
    cout << i << endl;
}
```

Each block has its own “Scope”, i.e., can define new variables (even with the same name). Variables defined within a block do not exist outside of it.

```
int main()
{
    int i = 0;
    {
        int i = 1;
        cout << i << endl;
    }
    cout << i << endl;
}
```

What is the output of this program?
"zero"

```
int main()
{
    int i = 0;

    if (i == 0)
        cout << "zero" << endl;
}
```


Basics

C++ Control Structures: IF – ELSE

What is the output of this program?
"zero"

```
int main()
{
    int i = 0;

    if (i == 0)
        cout << "zero" << endl;
    else
        cout << "not zero" << endl;
}
```

Basics

C++ Control Structures: IF – IF ELSE – ELSE

What is the output of this program?
“at least 50”

```
int main()
{
    int i = 50;

    if (i >= 50)
        cout << "at least 50" << endl;
    else if (i >= 40)
        cout << "at least 40" << endl;
    else if (i >= 30)
        cout << "at least 40" << endl;
    else if (i >= 20)
        cout << "at least 40" << endl;
    else if (i >= 10)
        cout << "at least 40" << endl;
    else
        cout << "between 0 and 9" << endl;
}
```

Basics

C++ Control Structures: IF – IF ELSE – ELSE

What is the output of this program?

“at least 50”

“at least 40”

“at least 30”

“at least 20”

“at least 10”

```
int main()
{
    int i = 50;

    if (i >= 50)
        cout << "at least 50" << endl;
    if (i >= 40)
        cout << "at least 40" << endl;
    if (i >= 30)
        cout << "at least 30" << endl;
    if (i >= 20)
        cout << "at least 20" << endl;
    if (i >= 10)
        cout << "at least 10" << endl;
    else
        cout << "between 0 and 9" << endl;
}
```

Elegant expression for:

if (i > 30)
 y="large";
else
 y="small";

```
int main()  
{  
    int i = 50;  
  
    string y = (i > 30) ? "large" : "small";  
    cout << y << endl;  
}
```

Basics

C++ Control Structures: Loops: WHILE

while (condition)

What is the last output?

10

```
int main()
{
    int i = 50;

    while (i > 10)
    {
        i--;
        cout << i;
    }
}
```

Basics

C++ Control Structures: Loops: DO – WHILE

**do {sth;}
while (condition);**
What is the last output?
10

```
int main()  
{  
    int i = 50;  
  
    do  
    {  
        i--;  
        cout << i;  
    } while (i > 10);  
}
```

Basics

C++ Control Structures: Loops: FOR

This produces the same result as the previous examples, right?
How many times is the loop executed?
51

```
int main()
{
    int i = 50;

    for (int y = 0; y <= 50; y++)
    {
        if (i > 10)
        {
            i--;
            cout << i;
        }
    }
}
```

Basics

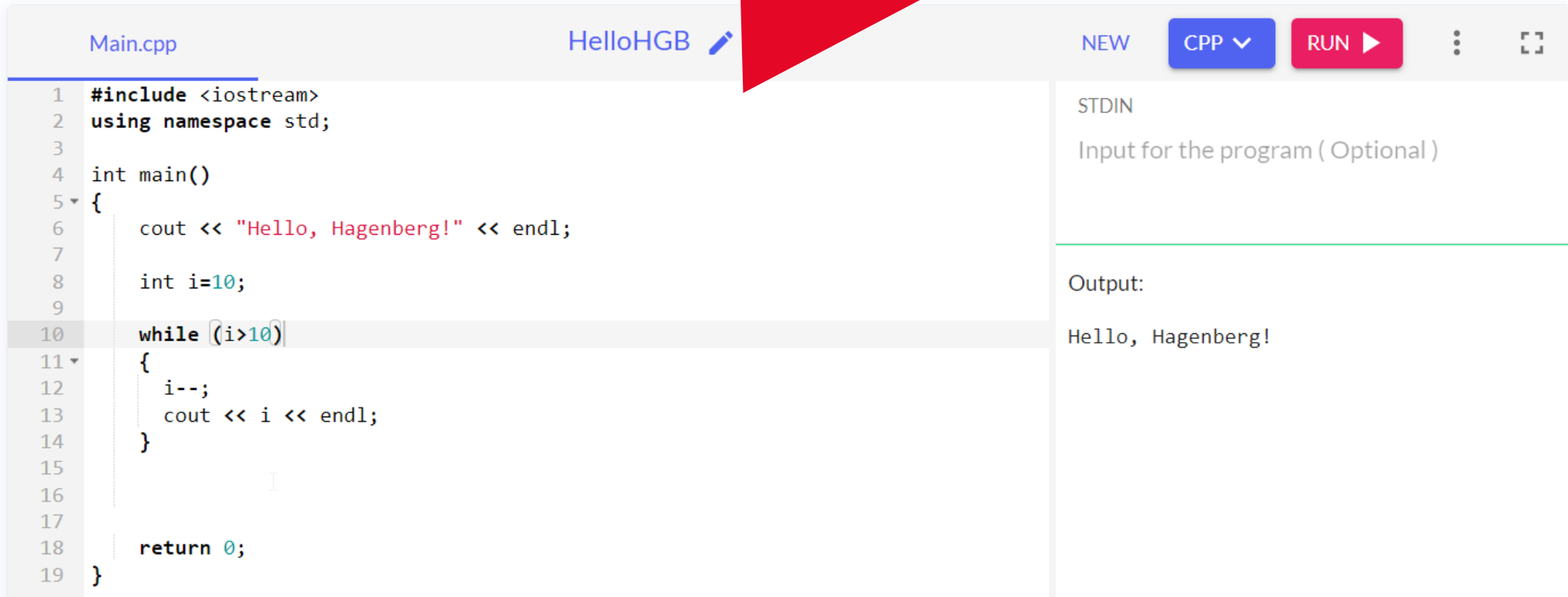
C++ Control Structures: Loops

Browse to: <https://onecompiler.com/cpp/3zq2dbjy9>

Modify to

- a do-while loop
- a for loop

Which questions arise (10mins)?



The screenshot shows an online C++ compiler interface. The top bar includes the filename 'Main.cpp', the user 'HelloHGB', and buttons for 'NEW', 'CPP', 'RUN', and a settings icon. The code editor displays the following C++ code:

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     cout << "Hello, Hagenberg!" << endl;
7
8     int i=10;
9
10    while ((i>10))
11    {
12        i--;
13        cout << i << endl;
14    }
15
16
17
18    return 0;
19 }
```

On the right side, the 'STDIN' section is labeled 'Input for the program (Optional)'. The 'Output' section displays the result of the program execution:

```
Output:
Hello, Hagenberg!
```


While the left program has no output
the right displays "9"
Beware about the loop body!

```
int i=10;  
while (i>10)  
{  
    i--;  
    cout << i << endl;  
}
```

```
int i=10;  
do  
{  
    i--;  
    cout << i << endl;  
} while (i>10);
```

Basics

C++ Control Structures: Loops: FOR

Better solution: Loop is only executed x times before the **break** tells us to jump out of the loop.

Better solution: while only!

```
int main()
{
    int i = 50;

    for (int y = 0; y <= 50; y++)
    {
        if (i > 10)
        {
            i--;
            cout << i;
        }
        else
            break;
    }
}
```

Basics

C++ Control Structures: Loops: FOR

In contrast to “break”, “**continue**” does not completely stop the loop, but just the **current iteration!**

1, 3, 5, 7, ...

```
int main()
{
    for (int y = 0; y <= 50; y++)
    {
        if (y % 2 == 0)
            continue;
        cout << y;
    }
}
```

Game Engine Architectures



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at